

SPORTZONE

Plataforma de E-commerce Esportivo de Alta Performance.
Arquitetura, Design e Engenharia de Qualidade.

1 Design System & Usabilidade

A identidade visual da SportZone foi concebida para transmitir **energia, foco e profissionalismo**. O sistema adota uma paleta *Dark Mode* complementada por um Laranja Vibrante, atraindo a atenção do usuário para as *Call To Actions* principais.

Laranja Primário
#F97316

Fundo Dark
#09090B

Superfícies
#27272A

**Sucesso
(Feedback)**
#22C55E

Componentização e Reaproveitamento

Toda a interface foi construída baseada na componentização do React aliada ao TailwindCSS para estilização. Os botões do e-commerce compartilham a mesma base visual, utilizando classes utilitárias. Os cards de produtos do *Grid* são renderizados dinamicamente a partir de um único template reutilizável, garantindo coesão visual máxima em todas as telas.

Btn Primário

**Btn
Secundário**

Feedback em Tempo Real

Para garantir uma excelente usabilidade (UX), o sistema atualiza o status do pedido em tempo real através de **WebSockets**. Micro-animações e mudanças de cor informam ao cliente o momento exato em que o pagamento é aprovado e o produto é despachado, sem recarregar a página.

2 Arquitetura e Implementação

O projeto segue uma arquitetura baseada em divisão de responsabilidades, separando rigidamente o cliente (Frontend) da lógica de negócios (Backend). A justificativa dessa escolha reside na **escalabilidade**: podemos evoluir o app Web sem impactar os serviços de pagamento.

Backend Robusto (Spring Boot)

Desenvolvido em Java 21 com Spring Boot 3. A estrutura do código é fortemente baseada no padrão MVC interno (Controller, Service, Repository), isolando a lógica de Domínio e garantindo que o acoplamento seja mantido no mínimo.

Frontend Moderno (React + TailwindCSS)

A interface foi construída adotando o ecossistema React para criar uma Single Page Application (SPA) robusta e escalável. O TailwindCSS foi escolhido para acelerar o desenvolvimento da UI através de classes utilitárias, garantindo que a renderização do catálogo de produtos seja responsiva e fluida.

3 Qualidade de Código: Padrão Strategy

Para o processamento de pagamentos, a escolha arquitetural foi o padrão comportamental **Strategy**. Ele nos permite introduzir novos métodos de pagamento (como Boleto ou Criptomoedas) sem modificar o serviço principal de pedidos, obedecendo ao princípio **Open/Closed (OCP)** do SOLID.

```
@Component
public class PagamentoStrategyFactory {

    private final Map<MetodoPagamento, PagamentoStrategy> strategies;

    @Autowired
    public PagamentoStrategyFactory(List<PagamentoStrategy> strategyList) {
        strategies = new EnumMap<>(MetodoPagamento.class);
        for (PagamentoStrategy strategy : strategyList) {
            strategies.put(strategy.getMetodo(), strategy);
        }
    }

    public PagamentoStrategy getStrategy(MetodoPagamento metodo) {
        return strategies.get(metodo);
    }
}
```

A injeção dinâmica no construtor faz com que qualquer nova classe que implemente **PagamentoStrategy** seja automaticamente registrada pelo Spring, eliminando longos blocos de **if-else** ou **switch-case**.

4 Processamento Assíncrono com Apache Kafka

O fluxo de compra tradicional costuma ser bloqueante. Na SportZone, decidimos por uma abordagem **Event-Driven (Orientada a Eventos)**.

Produtor (PedidoProducer)

Quando o cliente finaliza o checkout, o `PedidoService` apenas salva o estado inicial no banco e emite o evento `PedidoCriadoEvent` para o tópico do Kafka. O retorno para o frontend é imediato (non-blocking).

Consumidor (PedidoWorker)

Em background, o Worker intercepta o evento, delega para o Factory processar o pagamento via *Strategy*, simula a separação de estoque de forma isolada, e envia as notificações de sucesso via WebSocket para a interface gráfica.

Esta escolha garante **resiliência**. Se o serviço de pagamentos ficar lento ou fora do ar, o sistema não perde o pedido. A mensagem continua na fila do Kafka aguardando o sistema se recuperar.

5 Garantia de Qualidade (QA) e Cobertura

O plano de testes foi meticulosamente desenhado para focar estritamente nas regras de negócio. Ignoramos propositalmente DTOs, Entidades anêmicas e Getters/Setters auto-gerados (Lombok), focando nossos esforços nos algoritmos core.

Testes Unitários (Mockito)

As Factories e as Strategies foram validadas em completo isolamento, garantindo 100% de cobertura nos algoritmos de cálculo e seleção dinâmica.

Testes de Integração

Utilizamos `@EmbeddedKafka` para instanciar um broker em memória durante o build. O teste end-to-end comprova o envio pelo Producer e o consumo perfeito pelo Worker.

SportZone Backend

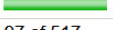
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.sportzone.controller		10%		n/a	4	6	16	18	4	6	0	2
 com.sportzone.kafka		80%		n/a	0	6	5	36	0	6	0	2
 com.sportzone		37%		n/a	1	2	2	3	1	2	0	1
 com.sportzone.service		100%		100%	0	9	0	43	0	8	0	1
 com.sportzone.strategy		100%		100%	0	12	0	34	0	10	0	3
Total	97 of 517	81%	0 of 6	100%	5	35	23	134	5	32	0	9

Figura 1: Evidência do sumário de cobertura de código do SportZone Backend (JaCoCo). Pacotes críticos de estratégia e infraestrutura apresentam alta taxa de validação.